The End of Programming

Matt Welsh

Communications of the ACM, January 2023

Paper Review by Taha Cheema - 21204729

Welsh's argument is primarily an argument of analogy. He gives the example of early computer scientists who worked closely with hardware and thought that a knowledge of semiconductors and logic gates would remain important for a computer scientist into the future. However, nowadays, CPU logic is all abstracted away from the programmer who never needs to operate with hardware. Similarly, Welsh claims that the next abstraction step will be one from programming to commanding AI agents. Programming will be akin to knowing how semiconductors and CPU cycles work, while manipulating AI agents will be the new abstraction computer scientists (if they can even be called that) will operate on. Welsh refers to this as humans' *relegation* to a mere *supervisory role*.

Welsh continues that the field of computer science will transition from an engineering field to an educative paradigm of "how to best instruct the machine" which itself has sufficient orchestration capacity and knowledge to put your instructions into action. His overarching claim is that this seismic shift presents "a huge opportunity, and plenty of huge risks".

My reflection of these arguments is that while Welsh correctly highlights how the field is evolving, he might be going too far with the argument of the atomic unit of computing changing from a computer system running an OS and programs to a massive AI model. I base this rebuttal on the principle of regulation. Having AI create and deploy entire software, especially security critical software, without any audit of the code (which Welsh considers something as obsolete as thinking about how a CPU is processing machine code instructions) is likely to be blocked by regulatory policies. We know that LLMs can hallucinate, and it is a mathematical inevitability that they do. Given that fact, a no-human-in-the-loop world where LLMs are simply commanded seems unlikely because governments and many organizations will simply not be willing to outsource control to a black box because the fallout from of a mistake or an unexpected AI takeover is huge. The breaking point in Welsh's analogy with hardware is as follows: as programmers, we moved past thinking about CPU cycles because the behavior of a CPU in processing instructions is predictable, visible, and repeatable. Code generation by LLMs is none of that. We could abstract away the CPU because it it was a deterministic process. We cannot abstract away code generated by LLMs because it is not deterministic.

Moreover, it helps to borrow a related concern raised by Epstein and Hertzmann about the future of art in an era of AI. They argue that AI art may become highly homogenized as future models years from now get trained on a huge swath of other AI-generated art, diminishing creativity and real character that humans produce in independent work. I can extend this to our discussion about programming. LLMs are a product of their training. If future models need to become these hyper-powerful, all-aware machines as per Welsh, they can't do so by training on more AI-generated code. And this is notwithstanding the fact that many of the world's software is proprietary and not trained on.

References

Ziv Epstein, Aaron Hertzmann. Art and the science of generative AI. Science 380, 1110-1111(2023). doi:10.1126/science.adh4451